

Training Deep Neural Network for Optimal Power Allocation in Islanded Microgrid Systems: A Distributed Learning-Based Approach

Fanghong Guo^{ID}, *Member, IEEE*, Bowen Xu^{ID}, Wen-An Zhang^{ID}, *Member, IEEE*,
Changyun Wen^{ID}, *Fellow, IEEE*, Dan Zhang^{ID}, *Senior Member, IEEE*,
and Li Yu^{ID}, *Senior Member, IEEE*

Abstract—Currently, numerical optimization methods are used to solve distributed optimal power allocation (OPA) problems for islanded microgrid (MG) systems. Most of them are developed based on rigorous mathematical derivation. However, the complexity of such optimization algorithms inevitably creates a gap between theoretical analysis and real-time implementation. In order to bridge such a gap, in this article we provide a new distributed learning-based framework to solve the real-time OPA problem. Specifically, inspired by the human-thinking scheme, distributed deep neural networks (DNNs) together with a dynamic average consensus algorithm are first employed to obtain an approximate OPA solution in a distributed manner. Then a distributed balance generation and demand algorithm is designed to fine-tune it to obtain the final optimal feasible solution. In addition, it is theoretically proved that the proposed DNN can well approximate one existing OPA algorithm (Guo et al. 2018), where quantitative numbers of at most how many hidden layers and neurons are provided. Several experimental case studies show that our proposed distributed learning framework can achieve similar optimal results to those obtained by using typical existing distributed numerical optimization methods while it is superior in terms of simplicity and real-time capability.

Index Terms—Bio-inspired optimization, deep neural network (DNN), distributed framework, learning to optimize, optimal power allocation (OPA).

I. INTRODUCTION

IN ORDER to meet the increasing electricity demand and power consumption need from remote rural areas and island residents, the concept of microgrid (MG) was proposed by Consortium for Electric Reliability Technology

Solutions (CERTS) at the beginning of this century [2]. According to its definition, an MG is a small-scale low-voltage power system that integrates power generation, transmission, distribution, and consumption together. Different from fossil-fueled thermal power generators in traditional power systems, most distributed energy resources (DERs) in MG system are powered by renewable energy resources such as photovoltaic and wind turbines. These DERs usually generate intermittent power, and they are also relatively small in terms of their power generation capability. Hence more advanced control and optimization techniques are needed for the energy management of MG system [3]–[5]. In [6]–[8], model predictive control (MPC) approaches are utilized in the energy management for MGs. Its main function is to provide real-time control of MG together with tracking optimal generation scheduling references calculated by some optimization algorithms. However, how to achieve real-time optimization to keep up with the pace of real-time control is still a very challenging topic.

For decades, distributed numerical optimization methods solving optimal power allocation (OPA) have been attracting a great deal of attention. The work in [9] proposes an alternating direction method of multipliers (ADMM) based distributed optimization algorithm. Li *et al.* [10] present a distributed interior point method using a θ -logarithmic barrier. In [11], a consensus-based distributed regularized primal-dual subgradient algorithm is developed to avoid projecting the estimates onto the global constraint set at every iteration but sacrifices a certain solution accuracy. Wang *et al.* [12] propose a distributed extremum-seeking algorithm based on average analysis. In [1], a hierarchical decentralized optimization architecture is proposed to solve the OPA, which requires gradient-based local optimization and an extra coordinator agent to conduct the global operation. In general, almost all distributed algorithms focus on the accuracy of the solution while ignoring the real-time requirements of today's distributed systems. In fact, they are suffering from high computational and communication costs. For example, the case studies in [10] show that it takes more than 10 s to solve a single distributed OPA problem on a 3-unit system. The reason behind this is that these distributed algorithms are almost all

Manuscript received May 13, 2020; revised November 5, 2020; accepted January 20, 2021. Date of publication February 10, 2021; date of current version May 3, 2022. This work was supported in part by the National Natural Science Foundation of China under Grant 61903333 and Grant 61822311, in part by the Zhejiang Qianjiang Talent Project under Grant QJD1902010, in part by the Key Research and Development Program of Zhejiang Province under Grant 2020C01109, and in part by the Open Research Project of the State Key Laboratory of Industrial Control Technology, Zhejiang University, China, under Grant ICT20089. (Corresponding author: Wen-An Zhang.)

Fanghong Guo, Bowen Xu, Wen-An Zhang, Dan Zhang, and Li Yu are with the Department of Automation, Zhejiang University of Technology, Hangzhou 310014, China (e-mail: fhguo@zjut.edu.cn; wazhang@zjut.edu.cn).

Changyun Wen is with the School of Electrical and Electronics Engineering, Nanyang Technological University, Singapore (e-mail: ecywen@ntu.edu.sg).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TNNLS.2021.3054778>.

Digital Object Identifier 10.1109/TNNLS.2021.3054778

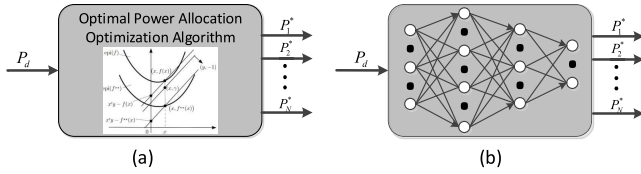


Fig. 1. Comparison illustration between the traditional optimization algorithm and learning-based algorithm. (a) Traditional optimization algorithm. (b) Learning-based algorithm.

gradient-based, which indicates that at least tens to thousands of iterations are required while *each iteration* contains at least one consensus operation. What's worse, each consensus operation contains dozens to thousands of communication and computing operations. As the scale of the system increases, such algorithms that require a large number of iterations will become increasingly inefficient.

Moreover, some distributed methods based on reinforcement learning (RL) have also been proposed to solve OPA problems in real-time. In [13], a distributed Q-learning-based algorithm is introduced to solve the OPA and unit commitment problems. Dai *et al.* [14] present a distributed RL algorithm for real-time OPA with unknown generation cost functions. The work in [15] develops a consensus-based distributed RL approach for OPA in MGs. In general, the above-mentioned RL algorithms treat the input of an OPA problem (such as a specific load) as a state and the corresponding optimal solution as an action. Under this setting, an OPA problem is transformed into a state-action pair. However, the state-action pair is a discrete case, which means that the above-mentioned RL algorithms are almost impossible to learn all feasible state-action pairs representing the whole relationship space of an OPA problem. In other words, these RL algorithms lack the necessary generalization capabilities so that they can only respond to several trained cases in real-time, which is also clearly shown in their case studies [13]–[15]. Therefore, when a continuous time-varying load is taken into account, such RL algorithms seem unable to solve the OPA problem in a distributed framework in real-time.

Let us reconsider the OPA problem in a different way. It is, in fact, a problem of trying to seek its optimal solution when total load demand is given. From the system point of view, an OPA optimization algorithm is indeed a system with the input being the total load demand P_d and the output being the optimal power generation from each dispatchable generator (DG), i.e., P_i^* , $\forall i$, as illustrated in Fig. 1(a). All existing algorithms are just doing one thing, that is to calculate P_i^* (output) based on a given P_d (input) by using various numerical approaches. Inspired by recent progress in machine learning technologies [16]–[19], it is natural to ask the following question: *Can we treat the existing OPA optimization algorithm as a “black box” and try to learn the relation between its input and output?* The answer should be positive. In fact, there are some pioneering works employing artificial neural networks to solve the OPA problem, which can be dated back to the 1990s, see [20], [21]. However, due to the limited computational resources at that time, only one hidden layer is considered in those artificial neural networks, which inevitably degrades the

approximation accuracy. Also, we should mention some works using DNN for real-time control of wireless resources [22] and landings [23]–[25]. However, the above approaches are all designed in a centralized way. As pointed out in [21], such a centralized mechanism may suffer from single-point failure and inflexibility to MG system re-configuration. Besides, its individual computational cost during the training stage would be very high when the system scales up.

To address these concerns, we propose a new *distributed* learning-based optimization framework to solve the OPA problem of an MG system in *real time*. The key idea behind this approach is to treat the solution resulted from a traditional OPA optimization algorithm as an unknown nonlinear mapping from the input P_d to outputs P_i^* , $\forall i = 1, \dots, N$. Based on the properties of neural networks, this mapping can be approximated by a deep neural network (DNN), as shown in Fig. 1(b). As long as the constructed neural network can accurately learn such nonlinear mapping, then the OPA problem is solved. Besides, different from the existing centralized network structure, a new distributed learning-based optimization framework is proposed, where a single large-scale DNN is decomposed into several small-scale distributed DNNs. Each local DNN is only responsible to learn the relationship between certain local information input and local output P_i^* . Intuitively, if such a DNN-based framework can well approximate an OPA optimization algorithm, e.g., the approach in [1], then it would be quite computationally cheap by simply passing the given load information to produce the optimal power output P_i^* . Compared to conventional optimization algorithms, the proposed learning-based approach only requires simple matrix-vector multiplication, which could significantly benefit us in computational time saving and thus is quite suitable for real-time OPA. In addition, it is worthy to point out that training such a network is also fairly cheap since the training samples can be easily obtained by running the optimization algorithm to be learned.

The main contributions of this article can be summarized as follows.

- 1) A new bio-inspired idea (from human thinking) for solving optimization problems is proposed. When human beings explore unknown problems, firstly they usually use their own “experience library” to form a rough judgment, and then make precise decisions with meticulous operations. Inspired by this scheme, similarly, the proposed approach uses DNN as “experience library” to quickly find approximate solutions firstly, and then use a specific algorithm to fine-tune approximate solutions to optimal solutions in a very short time.
- 2) Different from some existing work employing the DNN technique for real-time optimization/control in other areas in a centralized way, we first propose *distributed DNNs* to cooperatively solve an optimization problem in a distributed manner.
- 3) The *real-time distributed* solution of the OPA problem is given in this article, which, to the best of our knowledge, has not been well solved by any existing studies.
- 4) The dynamic generator status is also well tackled by our DNN-based approach, which is rarely considered

in the existing power/energy management and control approaches via the learning-based method.

The rest of this article is organized as follows. Section II mainly introduces the problem formulation. Before presenting the main results in Section IV, some preliminary knowledge on dynamic average consensus (DAC) algorithm, and a balance generation and demand algorithm is reviewed in Section III. Several experiment case studies are provided in Section V to validate the effectiveness of the proposed approach. At last, some concluding remarks are given in Section VI.

II. OPTIMAL POWER ALLOCATION IN MG SYSTEM

A. Problem Formulation

OPA in MGs aims to allocate the power output among DGs at lowest generation cost subject to the total load demand constraint and local generation constraints. It is usually formulated as an optimization problem as follows [26]:

$$\begin{cases} \min_P \sum_{i \in DG} f_i(P_i) \\ \text{s.t.} \sum_{i \in DG} P_i = P_d \\ P_i^{\min} \leq P_i \leq P_i^{\max} \quad \forall i \in DG \end{cases} \quad (1)$$

where DG denotes the set of DGs in an MG system, f_i is the generation cost function for the i th DG, which is usually approximated by the following quadratic function

$$f_i(P_i) = \alpha_i P_i^2 + \beta_i P_i + \gamma_i \quad (2)$$

with α_i , β_i , and γ_i being positive coefficients of the cost function, P_d is the total net load demand required for dispatchable generators (i.e., P_d equals to total load demand subtract the outputs from renewable energy sources), $P_i^{\min}$ and $P_i^{\max}$ denote the lower and upper bounds of the local DG respectively.

Remark 1: Note that in the OPA problem, only the power outputs of dispatchable generators are considered as the optimization variables. The non-DGs such as photovoltaic cells and wind turbines are usually treated as negative load and hence their power outputs can be lumped into the total load demand [27].

B. Limitations of Conventional Numerical Optimization Algorithms

In solving the OPA problem, the main concerns of existing distributed numerical optimization algorithms, such as those in [1], [26], [28], and [29], include high computational cost and slow convergence rate. On the other hand, the intermittent property of renewable energy sources in an MG system results in rapid and unpredictable fluctuations, which obviously demand a fast OPA algorithm to give real-time OPAs for such a system. Clear, it is very challenging for the existing numerical optimization algorithms to meet the requirement. To address such a challenging issue, we aim to re-solve the OPA problem in a learning-based strategy.

III. PRELIMINARY

Before presenting our main results, some preliminary knowledge on the DAC algorithm and balance generation and demand algorithm is introduced.

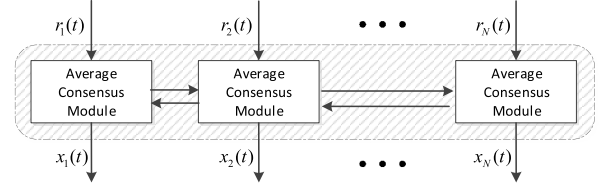


Fig. 2. Illustration of DAC algorithm.

A. Discrete-Time Dynamic Average Consensus Algorithm

The DAC algorithm is designed to allow a group of agents to track the average of their reference inputs in a distributed manner. Different from conventional static average consensus algorithms where the initial values are set as static constants, the reference inputs in the DAC algorithm are time-varying [30].

In this article, the first-order discrete-time DAC algorithm proposed in [30] is utilized to help us discover the time-varying total load demand P_d in a fully distributed way. The communication among N DGs is illustrated in Fig. 2 with $N = |DG|$ being the number of all the DGs.

Suppose the reference signal for the i th DG is denoted as $r_i(t)$, then $\Delta r_i(t)$ is the incremental change of $r_i(t)$ in the most recent sample

$$\Delta r_i(t) = r_i(t) - r_i(t-h), \quad t \geq 0 \quad (3)$$

where h is the time discretization unit, and $\Delta r_i(0) = r_i(0)$.

Denote the state of the i th DG at time instant t as $x_i(t)$. Then the DAC protocol can be written as

$$x_i(t+h) = x_i(t) + \sum_{j=1, j \neq i}^N a_{ij}(x_j(t) - x_i(t)) + \Delta r_i(t) \quad (4)$$

with a_{ij} being the interacting gain between the i th and j th DG. It is proved in [30] that under the DAC protocol in (3)–(4), all the DGs can track the average of their input references asymptotically, that is,

$$\lim_{t \rightarrow \infty} x_i(t) = x_j(t) = \frac{\sum_{i=1}^N r_i(t)}{N} \quad \forall i, j = 1, \dots, N. \quad (5)$$

If $r_i(t)$ is the local load consumption at the i th DG at time t , we can take advantage of the discrete-time DAC algorithm (3) and (4) to realize the global load consumption discovery in a distributed way.

B. Balance Generation and Demand Algorithm

In order to meet the balance requirement between DG generation and load demand, a distributed balance generation and demand algorithm is proposed in [15]. The main idea of this algorithm is reviewed as follows.

For the i th DG, denote the local power output and load as O_i and L_i respectively. By applying the following strategy, each DG can detect the average value of power-demand mismatch

$$\Delta_i[n+1] = \Delta_i[n] + \varsigma \sum_{j \neq i}^N a_{ij}(\Delta_j[n] - \Delta_i[n]) \quad (6)$$

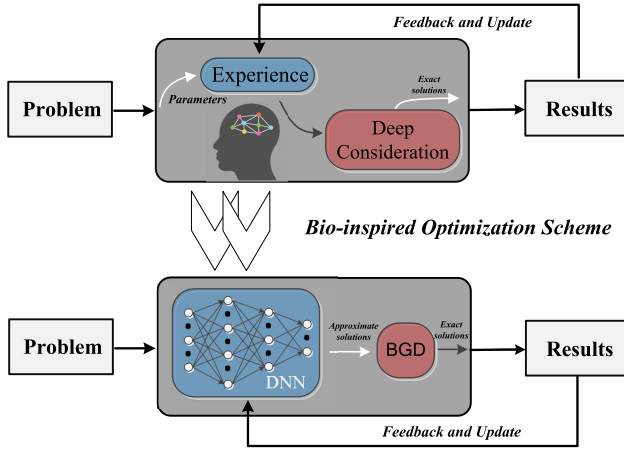


Fig. 3. Bio-inspired optimization scheme design.

where ς is a common step-size and $\Delta_i[0] = L_i - O_i$ is defined as the generation demand mismatch in the i th DG bus. Then under the protocol (6), one has

$$\Delta_i[n] \rightarrow \delta \triangleq \sum_{i=1}^N (L_i - O_i)/N$$

as $n \rightarrow \infty$.

In order to make the whole system generation and demand balance, the i th DG can revise its power output P_i by repeatedly computing

$$P_i = O_i + \delta \quad (7)$$

and

$$O_i = \begin{cases} P_i^{\max}, & \text{if } P_i > P_i^{\max} \\ P_i^{\min}, & \text{if } P_i < P_i^{\min} \\ P_i, & \text{otherwise} \end{cases} \quad (8)$$

until the balance between generation and demand is reached, that is $\delta = 0$. The effectiveness of this algorithm has been validated in [13].

IV. NEW DISTRIBUTED LEARNING BASED OPTIMIZATION APPROACH

In this section, to provide a distinguished real-time OPA solution for islanded MG system, a new distributed learning-based optimization approach is developed based on previous reviewed techniques.

A. Bio-Inspired Algorithm Design

As illustrated in Fig. 3, when human beings explore unknown problems, firstly they usually use their own “experience library” to form a rough judgment, and then make precise decisions with meticulous operations. Inspired by the above scheme, in this article, we propose a two-stage DNN-based optimization algorithm. In the first stage, the proposed algorithm uses DNN that acting as “experience library” to quickly find an approximate solution, and then utilizes a BGD algorithm to fine-tune the approximate solution to the optimal feasible solution in a very short time.

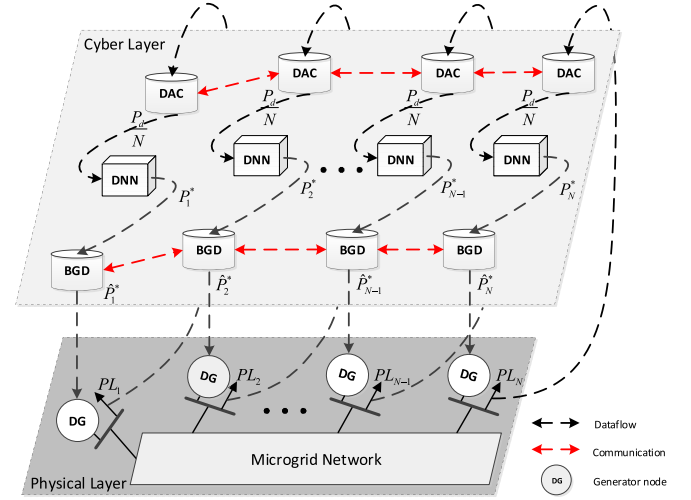


Fig. 4. Distributed learning based optimization approach for real-time OPA in MG.

B. Algorithm Structure

The proposed algorithm structure is presented in Fig. 4, which is designed with the aid of the two-layer multiagent framework adopted from [31]. The entire algorithm is located in the cyber layer, which is developed based on the idea of “horizontal-distributed and vertical-hierarchical.” Specifically, as shown in Fig. 4, the whole algorithm is decomposed among all the DGs, and each sub-algorithm consists of three cascading modules, i.e., DAC, DNN, and balance generation and demand (BGD) algorithm. By allowing all the sub-algorithms to communicate with their immediate neighbors, all the DGs in the physical layer can cooperatively solve the OPA problem formulated in (1). In practice, distributed communication in MG systems can be applied to wireless networks, such as Zig-Bee, WiFi, and cellular communication networks [32], [33].

The main procedure for each subalgorithm can be summarized as the following three steps.

1) *Step 1: Discover Global Load Demand by DAC*: The DAC module is designed in the first layer of our framework, as shown in Fig. 4. At each time instant t , every DAC module works as follows: **S1**) Synchronously monitor and measure the local load profile, i.e., $PL_i(t)$; **S2**) Communicate with neighboring DAC modules; **S3**) Compute and update the average load demand $(P_d(t)/N) = (\sum_{i=1}^N PL_i(t)/N)$ according to (3) and (4) by substituting $\Delta r_i(t)$ with $\Delta PL_i(t)$, where $\Delta PL_i(t)$ is defined as the incremental load change for i th DG, i.e., $\Delta PL_i(t) = PL_i(t) - PL_i(t-h)$.

2) *Step 2: Obtain Optimal Power Solution From DNN*: Once the average load demand $(P_d(t)/N)$ in Step 1 is ready, $(P_d(t)/N)$ is then directly fed into the well-trained local DNN network to extract the optimal power solution P_i^* for i th DG. Here the local DNN network plays a key role in approximating the following non-linear mapping relationship: $(P_d(t)/N) \rightarrow P_i^*$.

3) *Step 3: Adjust OPA Solution Through BGD*: Note that in Step 2, the optimal power output for the i th DG P_i^* is obtained from the local DNN network in a decentralized manner, which may slightly conflict with the demand-supply equality

constraint and local generation constraint. Hence some further adjustment for OPA solution is required in this step. Motivated by the work in [15], in this article a balance generation and demand algorithm as presented in (6)–(8) is utilized. The purpose of employing such an algorithm is to make the optimal solution obtained by DNN to be feasible.

Remark 2: The reason we select DAC rather than conventional static average consensus algorithm is that DAC allows a group of agents to track the average of their reference time-varying inputs in a distributed manner [30], which perfectly meets our requirement for real-time capability.

Remark 3: It is worthy to emphasize again that even though the problem (1) we formulate is a typical convex optimization problem, under the distributed framework, almost all existing distributed algorithms suffer from high computational and communication costs. The reason behind this is that these distributed algorithms are designed in an iterative way whereas each iteration requires at least one consensus operation containing dozens to thousands of communication and computing operations [1], [9]–[12]. As the scale of the system increases, such algorithms will become increasingly inefficient. In contrast, our DNN-based distributed algorithm only requires three steps including two consensus operations (DAC + BGD) to obtain the optimal solution in a distributed way. Therefore, from the perspective of algorithm design, our proposed algorithm is considerably superior.

C. DNN Network Training

DNNs play the role of “experience library” in our framework, thus one key step before algorithm implementation is to train the local DNN networks off-line to let them well approximate the nonlinear mapping relationship from $(P_d(t)/N)$ to P_i^* . In this subsection, a detailed DNN network training procedure is provided, which includes data set generation, training and validation, and testing stages.

1) *Data Set Generation:* In this stage, the training examples are generated by running a traditional numerical optimization algorithm, e.g., the hierarchical decentralized optimization (HDO) algorithm in [1] under different load demand profile P_d . The obtained optimal power output $P_i^{*(k)}$, $i = 1, \dots, N$ together with its corresponding average load demand $(P_d/N)^{(k)}$ constitute the k th training sample. It is worth mentioning that the range of training set should cover all the possible values of P_d , so that in the subsequent online application stage, there is no need to worry about the generalization performance of the well-trained DNN. Repeating running the HDO algorithm with arbitrary distinct load demand $P_d^{(k)}$, we can finally get the entire data set $\{((P_d/N)^{(k)}, P_i^{*(k)})\}_{k=1}^K$ for the i th DG. According to the common practice in neural network training [22], we randomly divide the data set into a training set, a validation set, and a test set at a ratio of 5:2:3. Note that all the training data have been normalized into $[-1, 1]$ and the corresponding normalization is also applied to the testing data.

2) *Training and Validation Stage:* We first use the training data set to optimize the weights of the DNN so that it gradually approaches our desired output. The cost function we use is the

mean square error (MSE) between label $\{P_i^{*(k)}\}$ and the output of the DNN. The optimization algorithm utilized for training here is *Adam*, which is most commonly used in deep learning recently [34]. Proper learning rate and batch size need to be selected by cross-validation. To further improve the training performance, the weights are initialized by using the truncated normal distribution.

3) *Testing Stage:* Once the training stage is completed, the DNN testing begins. The whole testing process for the DNN network is quite simple. Firstly, distinct average load demands (P_d/N) are fed into the well-trained DNN and collect its outputs, i.e., P_i^* . Then we compare the outputs obtained by the traditional HDO algorithm and the proposed DNN-based method to evaluate the accuracy of our proposed DNN-based architecture. MSE between the testing data and network output is used as an indicator to measure the network approximation accuracy. The smaller the MSE is, the more accurate the approximation is. If MSE is larger than a pre-specified threshold, we will return to the training stage and increase the number of hidden layers and their corresponding neurons. Besides, changing the activation function could also be an alternative solution to improve the approximation accuracy. Once the required accuracy is met, we focus on the calculation speed of the DNN-based framework to verify whether it is capable of conducting real-time OPA.

D. Algorithm Implementation

The proposed distributed learning-based optimization algorithm can be implemented and calculate the optimal power output for each DG in real-time, once the off-line training of the DNN network completes. To validate the effectiveness of our proposed scheme, several case studies will be provided in the next section.

Remark 4: The proposed distributed learning-based OPA algorithm distinguishes from traditional numerical optimization approaches in the following aspects: 1) no new mathematical derivation of the optimization algorithm is required, as we can use the well-established algorithm to generate the training data set. Instead, we can immediately get P_i^* by feeding $(P_d(t)/N)$ into the well-trained DNN network, where just some simple matrix calculations are manipulated; 2) the DNN-based method is a purely data-driven approach. If these optimal power outputs and corresponding load demand data are available, we even do not require to know the coefficients of cost functions in (1); and 3) the most significant advantage of the learning-based approach over traditional numerical optimization algorithms is that it requires less time to obtain the optimal solution online, which is quite suitable for real-time optimization.

Remark 5: It is worth pointing out that there are several other DNN architectures such as convolutional neural network (CNN) [35], long short-term memory (LSTM) recurrent neural network [36], autoencoder [37], and graph neural network (GNN) [38]. These architectures have their own advantages on feature extraction, classification, and prediction. The reason why we choose a fully connected DNN is that there lacks a special structure of the features of our formulated OPA

problem that can be exploited for such advanced feature extraction means including convolution and graph embedding. Therefore, considering that sophisticated feature extraction methods seem unhelpful, which runs counter to our original intention of pursuing efficiency and real-time capability, it is clear that a concise fully connected DNN [39] will be our first choice in this work.

E. Theoretical Analysis in DNN Approximation

It has been revealed in many existing works (see [18], [22], [39], [40], [41]) that DNN can be utilized to approximate certain kinds of nonlinear functions and some “learnable” iterative algorithms. However, it remains unclear whether the proposed DNN can well approximate the existing OPA algorithms. In addition, it is also unclear that the well-approximated DNN is a finite-size network or not. In other words, at most how many hidden layers and neurons are needed for such DNN to approximate the OPA algorithm within given approximation accuracy. In this subsection, based on the HDO algorithm in [1], we try to theoretically analyze the DNN approximation accuracy by giving quantitative numbers of the hidden layers and their neurons, which is summarized in the following Theorem.

Theorem 1: Consider a well-trained DNN with the structure presented in Section IV-B, and the training strategy illustrated in Section IV-C. It can well approximate Algorithm 1 (HDO algorithm) with no errors by employing $\mathcal{O}(T)$ hidden layers and $\mathcal{O}(NT)$ neurons, where T is the iteration step needed for HDO algorithm, and N is the total number of DGs.

Proof: See the Appendix. ■

Remark 6: It is worth pointing out that Theorem 1 gives the upper bound of the required number of hidden layers and their respective neurons. In practice, it is usually observed that much smaller networks are also able to achieve good approximation accuracy, which will be verified by the case studies in the next section.

V. CASE STUDIES

In this section, first, a 6-bus islanded MG system is employed to verify the performance of the proposed algorithm. Two case studies with and without generation constraints under static load demand are conducted to check the approximation accuracy of the proposed algorithm. Then a dynamic load profile is utilized to validate the on-line OPA ability of the proposed approach. At last, an IEEE 118-bus power system is adopted to exam our proposed approach by also taking the power loss into account. Three large-scale power systems consisting of 100, 500, 1000 DGs are designed respectively to compare the real-time performance between our proposed one and traditional numerical algorithms including hierarchical decentralized optimization (HDO) algorithm in [1] and ADMM algorithm in [42] in terms of the algorithm calculation time.

A. Experiment Setup

1) **Tested MG System:** In order to test the effectiveness of the proposed real-time OPA algorithm, an islanded MG system (shown in Fig. 5) is designed as a test system. There are

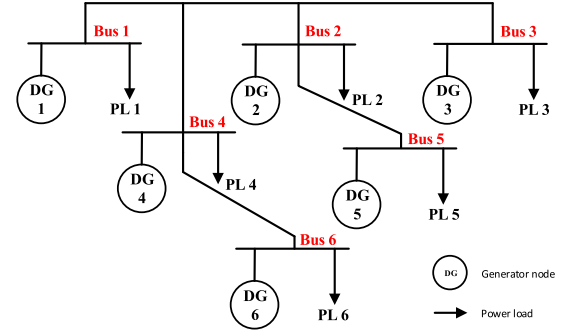


Fig. 5. 6-bus islanded MG test system.

TABLE I
GENERATOR PARAMETERS IN 6-BUS POWER SYSTEM

DG	α_i [\$/MW ² h]	β_i [\$/MWh]	γ_i [\$/h]	$P_i^{\min}$ [MW]	$P_i^{\max}$ [MW]
1	0.00375	2	0	50	200
2	0.0175	1.75	0	20	80
3	0.0625	1.0	0	15	50
4	0.00834	3.25	0	10	35
5	0.025	3.0	0	10	30
6	0.025	3.0	0	12	40

six dispatchable DGs in this system, and their parameters are adopted from [1], which are also listed in Table I. In addition, the nominal total load demand is set as $P_d = 283.4$ MW.

2) **Algorithm Setup:** In the following case studies, we employ the HDO algorithm proposed in [1] to generate the training examples. For the purpose of algorithm training, the total load demand P_d is assumed to be randomly changed to simulate the load fluctuation as well as the renewable sources penetration. In the following, two different cases, i.e., Case 1 without generation constraints, Case 2 with generation constraints, are considered. To fully cover normal range of P_d , for both Cases 1 and 2, P_d varies, respectively, in the ranges of [250 WM, 400 MW] and [141.7 MW, 425.1 MW], which covers the variations from 50% to 150% of its nominal value. The data sampling resolution for P_d is set as 0.01 MW in both cases. The minimum power adjustment in BGD is set as $s = 1e - 3$. Meanwhile, if $|\Delta(n + 1) - \Delta(n)| < 5e - 4$ is met, then we consider a balance between generation and demand is achieved.

For all our numerical results presented in this section, different from the theoretical results presented in Theorem 1, a much smaller DNN with one input layer, one output layer, and three hidden layers with each hidden layer containing 200 neurons are used, as shown in Fig. 6. In order to ensure the convergence of the DNN model, we set the training time to 1 h. After 1 h, training will stop automatically.

To find optimal parameters for our DNN, we perform cross-validation for this case study. Taking DG1 in Case 1 as an example, the impact of batch size and learning rate on the MSE evaluated on the validation data set early in the training is studied. As observed in Fig. 7(a), larger batch size leads to slower convergence, while smaller batch size incurs

TABLE II
OFF-LINE ACCURACY PERFORMANCE TEST OF PROPOSED ALGORITHM

MSE (DNN, HDO)	DG 1	DG 2	DG 3	DG 4	DG 5	DG 6	Average
Case 1	1.44e-6	1.03e-6	1.00e-6	1.12e-6	1.01e-6	1.01e-6	1.10e-6
Case 2	8.23e-4	3.80e-3	4.08e-3	2.17e-3	1.12e-3	4.20e-3	2.70e-3

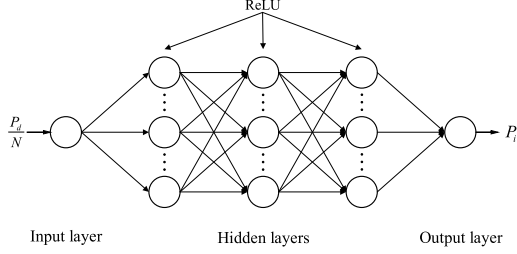


Fig. 6. DNN structure used in case studies.

TABLE III
OFF-LINE ACCURACY PERFORMANCE TEST OF PROPOSED ALGORITHM ($P_d = 283.4$ mW)

DG	Case 1			Case 2		
	HDO	DNN	error	HDO	DNN	error
1	189.3325	189.3326	1.80e-4	185.4036	185.4384	3.48e-2
2	47.7141	47.7133	7.70e-4	46.8722	46.8335	3.87e-2
3	19.3599	19.3590	9.29e-4	19.1242	19.1003	2.40e-2
4	10.1915	10.1910	4.38e-4	10.0000	10.0000	0
5	8.3999	8.3990	8.34e-4	10.0000	10.0000	0
6	8.3999	8.3990	8.32e-4	12.0000	12.0268	2.68e-2

non-convergence behavior. In addition, it is also observed from Fig. 7(b) that the convergence property does not change too much among the selected learning rate between 0.0005 and 0.005. Taking the above observations and practical concerns into consideration, we choose the batch size to be 1000 and the learning rate as 0.001.

The proposed DNN-based approach is implemented in Python 3.7.2 with TensorFlow 1.0.0 on a server computer with two 10-core Intel Broadwell processors, two Nvidia K40 Graphical Processing Units (GPUs), and 128 GB of memory. The GPUs are used in the training stage to reduce the training time, but are not used in the testing stage.

B. Off-Line Accuracy Performance Test

In this subsection, our new proposed framework is tested under static load demand P_d . We compare the results obtained by our proposed one with HDO algorithm in [1], which are shown in Figs. 8 and 9. It can be seen that in both cases, the mapping relationships between optimal power output and average load demand (P_d/N) $\rightarrow$ P_i^* can be well approximated by our proposed framework. As summarized in Table II, the average MSEs for two cases are 1.10e-6 and 2.70e-3, respectively, indicating that the estimation errors caused by our approach are negligible. In addition, with the total demand fixed as $P_d = 283.4$ MW, the approximation errors of the proposed DNN based approach are also listed in Table III. It can be observed that our proposed approach can achieve good approximation accuracy in both Cases 1 and 2.

C. On-Line Capability Performance Test

In this subsection, the on-line OPA ability of proposed approach is tested by considering dynamic load profile. For both cases, the load in Bus 1 is assumed to be time-varying while the others remain constant, that is,

$$PL_1(t) = 10 \sin(t/10) + 125, \quad PL_2(t) = 50, \quad PL_3(t) = 32.5 \\ PL_4(t) = 22.5, \quad PL_5(t) = 20, \quad PL_6(t) = 26.$$

We first intercept the output signal of the DAC within 120 steps to monitor the average consistency. As shown in Fig. 10, within 20 steps, the states of the six DGs reach a consensus, which implies that the implemented DAC algorithm makes a significant contribution to discovering the dynamic load demand in the whole dispatch framework.

Furthermore, during the next 100 time steps, we apply our algorithm to solve the OPA problem under the time-varying load condition. The experimental results are shown in Figs. 11 and 12, where the red line indicates the load demand profile of each DG bus while the blue line represents the real-time optimal power output. Both Figs. 11 and 12 have shown that our proposed distributed OPA algorithm is able to allocate the optimal power output for each DG in real-time.

D. Scalability Performance on Large-Scale Power Systems

In order to test the scalability of our proposed framework, firstly the IEEE 118-bus power system is chosen as a test system in this subsection, then three large-scale power systems consisting of 100, 500, and 1000 DGs are considered.

1) *IEEE 118-Bus Power System*: In this system, there are 54 dispatchable generators, whose cost function parameters are adopted from [43]. Meanwhile, the transmission power losses are also taken into consideration. Then (1) can be re-formulated as

$$\begin{cases} \min_P \sum_{i \in \text{DG}} f_i(P_i) \\ \text{s.t.} \sum_{i \in \text{DG}} P_i = P_d + L(P) \\ P_i^{\min} \leq P_i \leq P_i^{\max} \quad \forall i \in \text{DG} \end{cases} \quad (9)$$

where $L(P)$ represents the active power losses in the MG system. Similar to the results in [44], it just needs to meet a general assumption that $L(\cdot)$ is nonnegative, differentiable and satisfies

$$\gamma_i(P) = \frac{\partial L}{\partial P_i} \leq \gamma_0 < 1 \quad (10)$$

where γ_i denotes the fraction of an additional unit of power from the i th DG that is lost in the MG transmission lines. Similar to the results in [44], in this case study, γ_i is assumed to be varying randomly in the range [1%, 2%], which means that around 1%–2% of DGs' power output is lost in the MG transmission system.

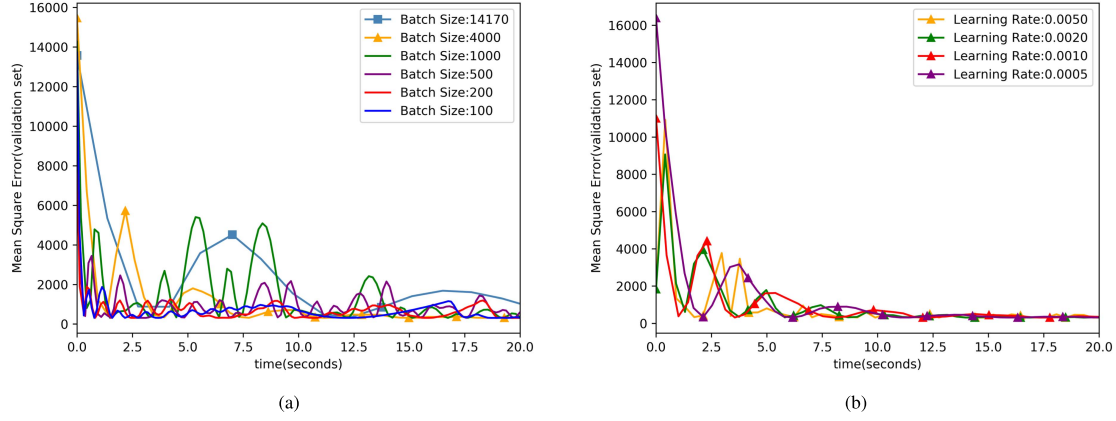


Fig. 7. Parameter selection for DG1 in Case 1. (a) Batch size selection. (b) Learning rate selection.

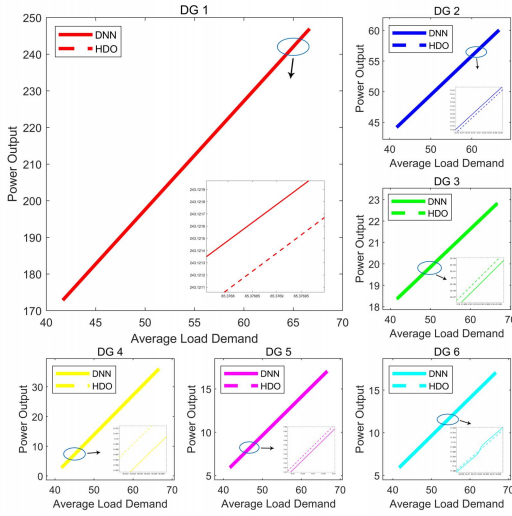


Fig. 8. Proposed framework accuracy performance test in Case 1.

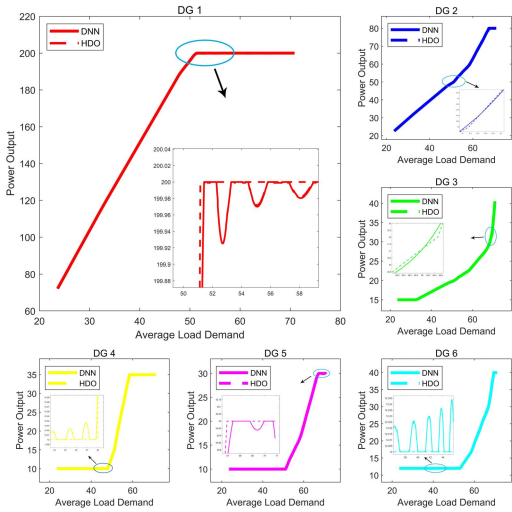


Fig. 9. Proposed framework accuracy performance test in Case 2.

In the training data generation step, we fix $\gamma_i = 1.5\%$ for all the DGs and let the total load demand P_d randomly varies in the range of [5000 MW, 15000 MW]. The data sampling

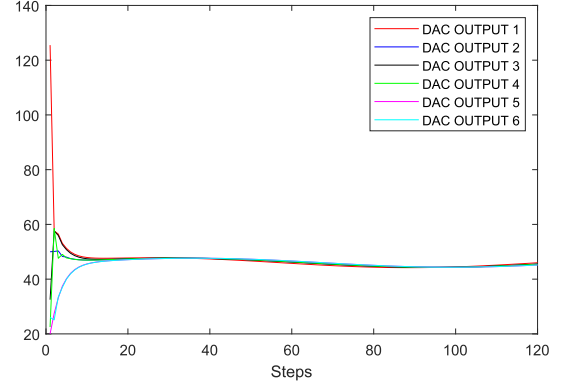


Fig. 10. DAC algorithm for time-varying load discovery.

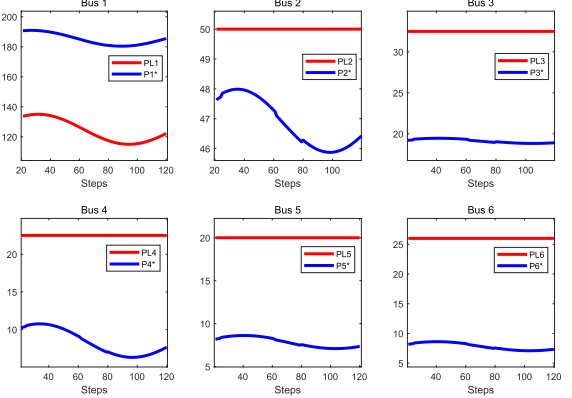


Fig. 11. On-line OPA of proposed learning based approach under time-varying load in Case 1.

resolution is set to 1 MW, which is much greater than that of the 6-bus MG system. The other experimental parameter settings remain the same as those in Cases 1 and 2.

To present the results more intuitively, we feed $P_d = 10000$ MW into our scheme and then compare its outputs with the results obtained by the HDO algorithm with random transmission power losses (namely, 1%–2% rule) and fixed power losses (namely, 1.5% rule) respectively. Besides, to better ascertain the robustness of our method to random power losses, extra test data are generated from the corresponding 1%–2% rule. The comparison results are

TABLE IV
OFF-LINE ACCURACY PERFORMANCE TEST OF PROPOSED ALGORITHM IN IEEE 118-BUS POWER SYSTEM ($P_d = 10000$ mW)

DG	P_i			error		DG	P_i			error	
	HDO(1-2%)	HDO(1.5%)	DNN	$ e_1 $	$ e_2 $		HDO(1-2%)	HDO(1.5%)	DNN	$ e_1 $	$ e_2 $
1	146.07	146.64	146.64	0.57	4.27e-4	28	448.04	448.26	448.26	0.22	2.23e-3
2	146.07	146.64	146.64	0.57	2.90e-4	29	449.27	449.49	449.49	0.22	2.11e-3
3	146.07	146.64	146.64	0.57	9.16e-5	30	591.99	592.28	592.28	0.28	8.54e-4
4	146.07	146.64	146.64	0.57	3.05e-4	31	146.07	146.64	146.64	0.57	2.90e-4
5	515.78	516.04	516.04	0.26	1.04e-3	32	146.07	146.64	146.64	0.57	2.14e-4
6	97.41	97.46	97.46	0.05	5.95e-4	33	146.07	146.64	146.64	0.57	5.65e-4
7	146.07	146.64	146.64	0.57	2.29e-4	34	146.07	146.64	146.64	0.57	5.19e-4
8	146.07	146.64	146.64	0.57	2.90e-4	35	146.07	146.64	146.64	0.57	2.75e-4
9	146.07	146.64	146.64	0.57	7.63e-5	36	146.07	146.64	146.64	0.57	6.56e-4
10	146.07	146.64	146.64	0.57	3.81e-4	37	546.79	574.06	574.06	0.27	6.71e-4
11	252.17	252.29	252.29	0.12	2.03e-3	38	146.07	146.64	146.64	0.57	4.27e-4
12	359.84	360.01	360.01	0.17	1.34e-3	39	4.58	4.59	4.59	0.00	6.45e-4
13	146.07	146.64	146.64	0.57	3.36e-4	40	695.85	696.20	696.20	0.35	2.87e-3
14	8.02	8.03	8.03	0.00	1.72e-4	41	146.07	146.64	146.64	0.57	5.19e-4
15	146.07	146.64	146.64	0.57	7.63e-5	42	146.07	146.64	146.64	0.57	3.05e-4
16	146.07	146.64	146.64	0.57	1.37e-4	43	146.07	146.64	146.64	0.57	3.66e-4
17	146.07	146.64	146.64	0.57	2.44e-4	44	146.07	146.64	146.64	0.57	3.36e-4
18	146.07	146.64	146.64	0.57	6.41e-4	45	288.83	288.97	288.97	0.14	7.63e-4
19	146.07	146.64	146.64	0.57	3.51e-4	46	45.84	45.87	45.87	0.02	1.98e-4
20	21.78	21.79	21.79	0.01	3.72e-4	47	146.07	146.64	146.64	0.57	1.83e-4
21	233.80	233.91	233.91	0.11	2.01e-3	48	146.07	146.64	146.64	0.57	2.29e-4
22	55.01	55.04	55.04	0.03	3.59e-4	49	146.07	146.64	146.64	0.57	5.19e-4
23	146.07	146.64	146.64	0.57	6.56e-4	50	146.07	146.64	146.64	0.57	5.04e-4
24	146.07	146.64	146.64	0.57	1.53e-4	51	41.26	41.28	41.28	0.02	3.81e-4
25	177.63	177.72	177.72	0.09	1.33e-3	52	146.07	146.64	146.64	0.57	1.83e-4
26	183.37	183.46	183.46	0.09	5.80e-4	53	146.07	146.64	146.64	0.57	4.57e-4
27	146.07	146.64	146.64	0.57	3.05e-4	54	146.07	146.64	146.64	0.57	3.51e-4
average error $ \bar{e}_1 $				0.42		average error $ \bar{e}_2 $				6.00e-4	

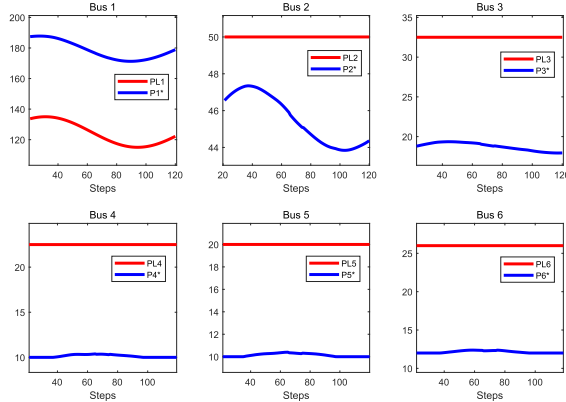


Fig. 12. On-line OPA of proposed learning based approach under time-varying load in Case 2.

listed in Table IV. It can be observed that our proposed approach achieves almost the same results as those of the HDO algorithm under 1.5% rule (i.e., with fixed 1.5% power loss) with the average error $|\bar{e}_2|$ being $6e-4$. In addition, the average error between outputs of DNNs and the HDO algorithm under the 1%-2% rule (i.e., with random power loss varying in the range [1%, 2%]) is $|\bar{e}_1| = 0.42$. This indicates that even if there are unavoidable random power losses in the MG system, all the generators can accurately estimate its optimal power output with our proposed scheme. Moreover, we also compare the computational time of our

proposed approach with the HDO algorithm. In this case study, it takes 12.76 s for the HDO algorithm, while our approach only needs 0.38 s. Therefore, it can be seen that our proposed method achieves excellent performances in terms of both approximation accuracy and computational efficiency.

2) *Three Large-Scale Power Systems*: Three large-scale power systems consisting of 100, 500, 1000 DGs are considered. The cost function parameters of these DGs are randomly generated in the following ranges, e.g., $\alpha_i \in [0.01, 0.02]$, $\beta_i \in [1, 2]$. The total load demand P_d is also randomly generated ranging from 80% to 120% of its nominal value P_d' . The data sampling resolution is set as 0.01 and we compare the calculation time of the proposed algorithm with traditional gradient-based HDO algorithm in [1] and ADMM algorithm in [42] subject to different power system scalability, which are summarized in Table V. Note that the HDO algorithm and ADMM algorithm are run in MATLAB R2018a while the DNN-based algorithm is in Python 3.7.2. All of them are conducted on the same server computer. It can be observed the CPU calculation time required by both the HDO and ADMM algorithms becomes longer and longer when the system scales up. However, our proposed framework still remains at a very low computational time regardless of the system size. In other words, the proposed learning-based optimization approach has tremendous advantages on real-time OPA, especially for a large-scale power system.

TABLE V
SCALABILITY PERFORMANCE TEST OF PROPOSED ALGORITHM

Case (P_d', N)	Total CPU time (sec.)			
	DNN (Python)	HDO (Matlab)	ADMM (Matlab)	DNN/HDO
(10000,100)	0.4325	20.7813	67.3539	2.08%
(50000,500)	0.7291	126.2656	294.9357	0.58%
(100000,1000)	0.8754	1081.1718	635.8829	0.08%

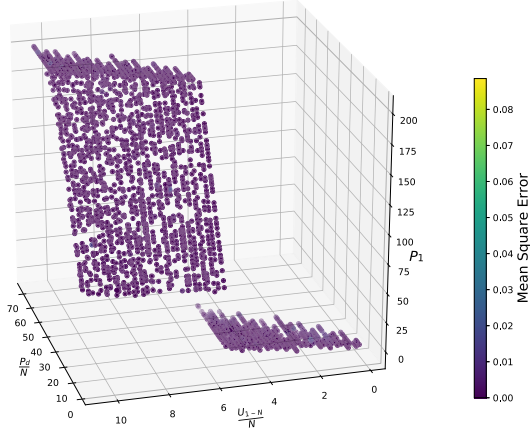


Fig. 13. Proposed framework accuracy performance test with dynamic generator status. There are 3000 test samples selected from the test set and visualized in the 3-D space with the average MSEs being $1.07\text{e-}3$.

E. Flexibility Performance on Dynamic Generators Status

Considering that in practical scenarios, some generators will be turned on/off for specific reasons [45], therefore, our designed algorithm should be able to adapt to different statuses of generators. Motivated by this consideration, the average status of generators ($U_{1\sim N}/N \triangleq (\sum_{i=1}^N 2^{N-1} U_N/N)$), together with the average total load demand (P_d/N), are taken as the input of DNN in this case study. Specifically, the generator status is first denoted as a set of binary variables $[U_N, \dots, U_1]$, where $U_i = 1$ indicates the i th generator is on, otherwise is off. Then each binary variable is converted into a decimal variable $[2^{N-1} U_N, \dots, 2^0 U_1]$. Finally, with such decimal variables as the initial state of the DAC algorithm, the average status of generators is obtained immediately and then fed into the local DNN.

The tested system is selected the same as that in Section V-A and the generation constraints are also considered. To build a sufficiently representative data set, $U_{1\sim N}$ traverses all feasible generator status and P_d is uniformly sampled in the corresponding feasible region. The training process stops when the average MSE value of the validation set is less than $1\text{e-}3$. The rest of the experimental settings are the same as those in Section V-A.

As shown in Fig. 13, there are 3000 test samples visualized in the 3-D space while each point represents an optimal solution obtained by the HDO algorithm. The color of each point denotes the corresponding MSE between the two solutions obtained by HDO and DNN respectively. It can be observed that our proposed approach generates satisfactory solutions with the average of MSEs being $1.07\text{e-}3$.

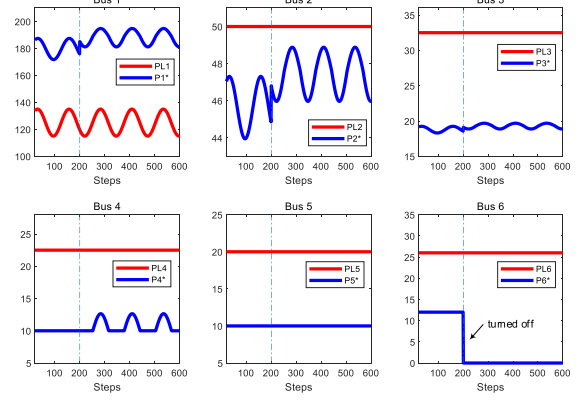


Fig. 14. On-line OPA of proposed learning based approach under time-varying load and dynamic generator status. The 6th generator is turned off at step 200.

To further validate the on-line OPA performance of the proposed learning-based framework, in addition to maintaining a dynamic load, we also turn off the 6th generator at Time Step 200 and plot the OPA results within 600 steps, as shown in Fig. 14. It can be observed that each generator in the 6-bus system achieves OPA in real-time which indicates that the DNN-based framework can perfectly adapt to dynamic generator status.

VI. CONCLUSION

In this article, a new distributed learning-based real-time optimization algorithm is proposed to allocate powers among all the dispatchable DERs in an islanded MG system. Under our proposed two-layer distributed framework, a DNN together with DAC, and balance generation and demand techniques are employed. Compared to traditional distributed numerical optimization algorithms, the proposed learning-based approach has the following advantages: 1) the optimal solution can be directly obtained by simply feeding the load demand, which takes much less computational time than traditional optimization algorithm and thus can achieve real-time optimization; 2) different from conventional single DNN network, the proposed distributed DNN structure is suitable for large-scale system optimization; and 3) dynamic generator status is also considered. The effectiveness of the proposed approach has been validated by several case studies implemented on a 6-bus islanded MG system and several large-scale MG systems.

It would be challenging but interesting to consider the following topics as future research: 1) how to take the energy storage system into account in the OPA problem formulation; 2) how to obtain the ground truth of a non-convex OPA problem; and 3) integration of renewable energy prediction with our proposed DNN based framework.

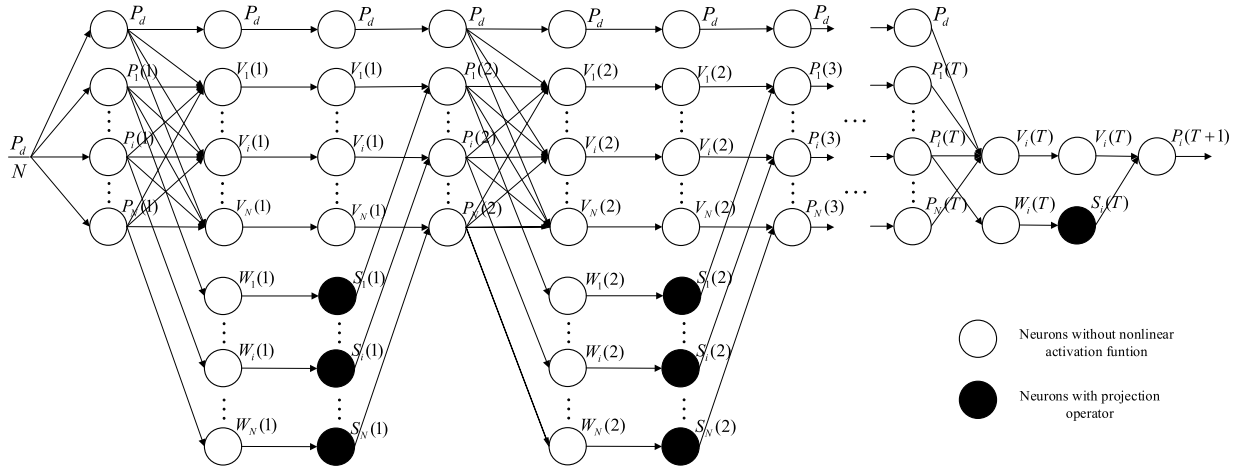


Fig. 15. Consecutive DNN structure for approximation of HDO algorithm in [1].

APPENDIX PROOF OF THEOREM 1

Proof: In this appendix, we aim to theoretically prove that DNN is capable of approximating the HDO algorithm proposed in [1]. For convenience, the pseudocode of HDO algorithm in [1] is summarized in Algorithm 1.

Algorithm 1 Pseudo Code of HDO Algorithm in [1]

Initialize the outputs $P_i(0) \in \mathbb{R}$

Set $l = 0$

repeat

Gradient-descent operation

$$W_i(l) = P_i(l) - \zeta_l \nabla f_i(P_i(l))$$

Local constraint projection

$$S_i(l) = P_{X_i}[W_i(l)] = [W_i(l)]_{P_i^{\min}}^{P_i^{\max}}$$

Global constraint projection

$$X_i(l) = P_{X_g}[P_i(l)] = P_i(l) - \frac{\sum_{i=1}^N P_i(l) - P_d}{n}$$

Coordinate operation

$$P_i(l+1) = \frac{S_i(l) + (N-1)P_i(l) + X_i(l)}{N+1}$$

Iteration update

$$l = l + 1$$

until Some stopping criteria are met

Output $P_i(l+1)$

Similar to the analysis procedure in [22], the main steps of the DNN approximation analysis are outlined as follows:

Step 1: Construct simple neural networks that may consist of ReLUs to represent the mapping between adjacent variables in one iteration;

Step 2: Compose these small neural networks to approximate a rational function representing one iteration of the algorithm;

Step 3: Concatenate these rational functions to approximate the entire algorithm;

Step 4: Analyze the error between the above network and traditional numerical calculation methods.

It is well known that only a combination of linear transformations and nonlinear activation functions such as ReLU units

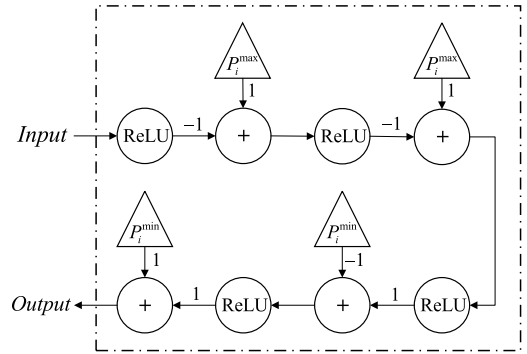


Fig. 16. Implementation of the projection operator.

can be implemented by DNN [18]. For Algorithm 1, we first focus on the transition between adjacent variables.

For the cost function given in (2), its gradient can be written as $\nabla f_i(P_i(l)) = 2\alpha_i P_i(l) + \beta_i$, which means $W_i(l)$ in Algorithm 1 can be represented by a linear function of $P_i(l)$. Further, the local constraint projection operator $[\cdot]_{P_i^{\min}}^{P_i^{\max}}$ can be implemented by a total of four ReLU units as illustrated below as well as in Fig. 16

$$[I]_{P_i^{\min}}^{P_i^{\max}} = P_i^{\min} + \max(\max(\zeta, 0) - P_i^{\min}, 0) \quad (11)$$

where ζ is the projection operator $[\cdot]_0^{P_i^{\max}}$ which can be implemented by two ReLU units

$$\zeta = [I]_0^{P_i^{\max}} = P_i^{\max} - \max(P_i^{\max} - \max(I, 0), 0) \quad (12)$$

where I is the input.

Once $S_i(l)$ is obtained, we need to analyze the implementation of $(n-1)P_i(l) + X_i(l)$, for convenience, which is denoted as $V_i(l)$

$$\begin{aligned} V_i(l) &\triangleq (N-1)P_i(l) + X_i(l) \\ &= NP_i(l) - \frac{\sum_{i=1}^N P_i(l) - P_d}{N}. \end{aligned} \quad (13)$$

Clearly, it shows that each $V_i(l)$ can be represented by a linear function of P_d and $P_i(l)$, $i \in 1, \dots, N$.

Finally we get $P_i(l+1)$ through a simple linear combination of $S_i(l)$ and $V_i(l)$ as

$$P_i(l+1) = \frac{S_i(l) + V_i(l)}{N+1}. \quad (14)$$

The whole implementation construction of the HDO algorithm by using DNN is shown in Fig. 15. The neurons drawn in black indicates that they use the projection activation function in (11), while the neurons drawn in white represent that they do not use any nonlinear activation function.

Based on the above construction, we can derive the required number of neurons and layers for approximating the HDO algorithm by counting those in Fig. 15. Suppose the total number of iterations in the HDO algorithm is denoted as T . First, we need $N+1$ neurons in the first layer to store Pd and all $P_i(1)$. In any iteration $l \in [1, T-1]$, we need 1 layer containing $2N$ neurons for all $V_i(l)$ and $W_i(l)$, 4 layers containing $4N$ neurons (with ReLUs) for all $S_n(l)$, and 1 layer containing N neurons for all $P_i(l+1)$. In the last iteration T , 6 layers, and 7 neurons are all we need. Thus, it is clear that we need $(6T+1)$ layers with $(7NT-6N+8)$ neurons to approximate Algorithm 1 without error theoretically.

This completes the proof. ■

REFERENCES

- [1] F. Guo, C. Wen, J. Mao, J. Chen, and Y.-D. Song, "Hierarchical decentralized optimization architecture for economic dispatch: A new approach for large-scale power system," *IEEE Trans. Ind. Informat.*, vol. 14, no. 2, pp. 523–534, Feb. 2018.
- [2] R. H. Lasseter, "Microgrids," in *Proc. IEEE Power Eng. Soc. Winter Meeting Conf.*, vol. 1, Jan. 2002, pp. 305–308.
- [3] H. Han, X. Hou, J. Yang, J. Wu, M. Su, and J. M. Guerrero, "Review of power sharing control strategies for islanding operation of AC microgrids," *IEEE Trans. Smart Grid*, vol. 7, no. 1, pp. 200–215, Jan. 2016.
- [4] G. Wen, G. Hu, J. Hu, X. Shi, and G. Chen, "Frequency regulation of source-grid-load systems: A compound control strategy," *IEEE Trans. Ind. Informat.*, vol. 12, no. 1, pp. 69–78, Feb. 2016.
- [5] D. Wang, Z. Wang, and C. Wen, "Distributed optimal consensus control for a class of uncertain nonlinear multiagent networks with disturbance rejection using adaptive technique," *IEEE Trans. Syst., Man, Cybern. Syst.*, early access, Aug. 30, 2019, doi: [10.1109/TSMC.2019.2933005](https://doi.org/10.1109/TSMC.2019.2933005).
- [6] Y. Zhang, F. Meng, R. Wang, B. Kazemtabrizi, and J. Shi, "Uncertainty-resistant stochastic MPC approach for optimal operation of CHP microgrid," *Energy*, vol. 179, pp. 1265–1278, Jul. 2019.
- [7] A. Parisio, C. Wiezorek, T. Kyneta, J. Elo, K. Strunz, and K. H. Johansson, "Cooperative MPC-based energy management for networked microgrids," *IEEE Trans. Smart Grid*, vol. 8, no. 6, pp. 3066–3074, Nov. 2017.
- [8] G. Lou, W. Gu, Y. Xu, M. Cheng, and W. Liu, "Distributed MPC-based secondary voltage control scheme for autonomous droop-controlled microgrids," *IEEE Trans. Sustain. Energy*, vol. 8, no. 2, pp. 792–804, Apr. 2017.
- [9] G. Chen and Q. Yang, "An ADMM-based distributed algorithm for economic dispatch in islanded microgrids," *IEEE Trans. Ind. Informat.*, vol. 14, no. 9, pp. 3892–3903, Sep. 2018.
- [10] C. Li, X. Yu, T. Huang, and X. He, "Distributed optimal consensus over resource allocation network and its application to dynamical economic dispatch," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 6, pp. 2407–2418, Jun. 2018.
- [11] D. Yuan, D. W. Ho, and S. Xu, "Regularized primal–dual subgradient method for distributed constrained optimization," *IEEE Trans. Cybern.*, vol. 46, no. 9, pp. 2109–2118, Sep. 2016.
- [12] D. Wang, M. Chen, and W. Wang, "Distributed extremum seeking for optimal resource allocation and its application to economic dispatch in smart grids," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 10, pp. 3161–3171, Oct. 2019.
- [13] F. Li, J. Qin, and W. X. Zheng, "Distributed Q -learning-based online optimization algorithm for unit commitment and dispatch in smart grid," *IEEE Trans. Cybern.*, vol. 50, no. 9, pp. 4146–4156, Sep. 2020.
- [14] P. Dai, W. Yu, G. Wen, and S. Baldi, "Distributed reinforcement learning algorithm for dynamic economic dispatch with unknown generation cost functions," *IEEE Trans. Ind. Informat.*, vol. 16, no. 4, pp. 2258–2267, Apr. 2020.
- [15] F. Li, J. Qin, Y. Kang, and W. X. Zheng, "Consensus based distributed reinforcement learning for nonconvex economic power dispatch in microgrids," in *Neural Information Processing (Series Lecture Notes in Computer Science)*. Berlin, Germany: Springer, Nov. 2017, pp. 831–839.
- [16] M. Eisen, C. Zhang, L. F. O. Chamon, D. D. Lee, and A. Ribeiro, "Learning optimal resource allocations in wireless systems," *IEEE Trans. Signal Process.*, vol. 67, no. 10, pp. 2775–2790, May 2019.
- [17] K. Li and J. Malik, "Learning to optimize," 2016, *arXiv:1606.01885*. [Online]. Available: <http://arxiv.org/abs/1606.01885>
- [18] S. Liang and R. Srikant, "Why deep neural networks for function approximation?" 2016, *arXiv:1610.04161*. [Online]. Available: <http://arxiv.org/abs/1610.04161>
- [19] Y.-D. Song, X. Huang, and Z.-J. Jia, "Dealing with the issues crucially related to the functionality and reliability of NN-associated control for nonlinear uncertain systems," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 28, no. 11, pp. 2614–2625, Nov. 2017.
- [20] J. Nanda, "Application of artificial neural network to economic load dispatch," in *Proc. Int. Conf. Adv. Power Syst. Control, Operation Manage. (APSCOM)*, vol. 2, 1997, pp. 707–711.
- [21] S. Panta, S. Premrudeepreechachani, S. Nuchprayoon, C. Dechthummarong, S. Janjornmanit, and S. Yachiangkam, "Optimal economic dispatch for power generation using artificial neural network," in *Proc. Int. Power Eng. Conf.*, Dec. 2007, pp. 1343–1348.
- [22] H. Sun, X. Chen, Q. Shi, M. Hong, X. Fu, and N. D. Sidiropoulos, "Learning to optimize: Training deep neural networks for interference management," *IEEE Trans. Signal Process.*, vol. 66, no. 20, pp. 5438–5453, Oct. 2018.
- [23] C. Sánchez-Sánchez and D. Izzo, "Real-time optimal control via deep neural networks: Study on landing problems," *J. Guid., Control, Dyn.*, vol. 41, no. 5, pp. 1122–1135, May 2018.
- [24] S. You, C. Wan, R. Dai, P. Lu, and J. R. Rea, "Learning-based optimal control for planetary entry, powered descent and landing guidance," in *Proc. AIAA Scitech Forum*, Jan. 2020, p. 0849.
- [25] L. Cheng, Z. Wang, Y. Song, and F. Jiang, "Real-time optimal control for irregular asteroid landings using deep neural networks," *Acta Astronautica*, vol. 170, pp. 66–79, May 2020.
- [26] F. Guo, G. Li, C. Wen, L. Wang, and Z. Meng, "An accelerated distributed gradient-based algorithm for constrained optimization with application to economic dispatch in a large-scale power system," *IEEE Trans. Syst., Man, Cybern. Syst.*, early access, Sep. 5, 2019.
- [27] S.-J. Ahn, S.-R. Nam, J.-H. Choi, and S.-I. Moon, "Power scheduling of distributed generators for economic and stable operation of a microgrid," *IEEE Trans. Smart Grid*, vol. 4, no. 1, pp. 398–405, Mar. 2013.
- [28] G. Chen, F. L. Lewis, E. N. Feng, and Y. Song, "Distributed optimal active power control of multiple generation systems," *IEEE Trans. Ind. Electron.*, vol. 62, no. 11, pp. 7079–7090, Nov. 2015.
- [29] C. Li, X. Yu, W. Yu, T. Huang, and Z.-W. Liu, "Distributed event-triggered scheme for economic dispatch in smart grids," *IEEE Trans. Ind. Informat.*, vol. 12, no. 5, pp. 1775–1785, Oct. 2016.
- [30] M. Zhu and S. Martínez, "Discrete-time dynamic average consensus," *Automatica*, vol. 46, no. 2, pp. 322–329, Feb. 2010.
- [31] F. Guo, C. Wen, and Y.-D. Song, *Distributed Control and Optimization Technologies in Smart Grid Systems*. Boca Raton, FL, USA: CRC Press, 2017.
- [32] H. Liang, B. J. Choi, W. Zhuang, and X. Shen, "Stability enhancement of decentralized inverter control through wireless communications in microgrids," *IEEE Trans. Smart Grid*, vol. 4, no. 1, pp. 321–331, Mar. 2013.
- [33] Y. Wang, Y. Song, M. Krstic, and C. Wen, "Fault-tolerant finite time consensus for multiple uncertain nonlinear mechanical systems under single-way directed communication interactions and actuation failures," *Automatica*, vol. 63, pp. 374–383, Jan. 2016.
- [34] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*. [Online]. Available: <http://arxiv.org/abs/1412.6980>
- [35] Y. Le Cun and Y. Bengio, "Convolutional networks for images, speech, and time series," in *The Handbook of Brain Theory and Neural Networks*, M. A. Arbib, Ed. Cambridge, MA, USA: MIT Press, 1995, pp. 255–258.

- [36] F. A. Gers, J. Schmidhuber, and F. Cummins, "Learning to forget: Continual prediction with LSTM," in *Proc. 9th Int. Conf. Artif. Neural Netw.*, 1999, pp. 850–855.
- [37] A. Ng, "Sparse autoencoder," *CS294A Lect. Notes*, vol. 72, pp. 1–19, 2011.
- [38] F. Scarselli, M. Gori, A. Chung Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, Jan. 2009.
- [39] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Netw.*, vol. 2, no. 5, pp. 359–366, Jan. 1989.
- [40] X. Le, S. Chen, Z. Yan, and J. Xi, "A neurodynamic approach to distributed optimization with globally coupled constraints," *IEEE Trans. Cybern.*, vol. 48, no. 11, pp. 3149–3158, Nov. 2018.
- [41] F. Liang, C. Shen, W. Yu, and F. Wu, "Towards optimal power control via ensembling deep neural networks," *IEEE Trans. Commun.*, vol. 68, no. 3, pp. 1760–1776, Mar. 2020.
- [42] W. Deng, M.-J. Lai, Z. Peng, and W. Yin, "Parallel multi-block ADMM with $o(1/k)$ convergence," *J. Sci. Comput.*, vol. 71, no. 2, pp. 712–736, 2017.
- [43] P. A. Lipka, R. P. O'Neill, and S. Oren, "Developing line current magnitude constraints for IEEE test problems optimal power flow paper 7," Tech. Rep., Apr. 2013. [Online]. Available: <https://www.ferc.gov/industries/electric/indusact/market-planning/opf-papers/acopf-7-line-constraints.pdf>
- [44] R. Mudumbai, S. Dasgupta, and B. B. Cho, "Distributed control for optimal economic dispatch of a network of heterogeneous power generators," *IEEE Trans. Power Syst.*, vol. 27, no. 4, pp. 1750–1760, Nov. 2012.
- [45] G. Wen, X. Yu, Z.-W. Liu, and W. Yu, "Adaptive consensus-based robust strategy for economic dispatch of smart grids subject to communication uncertainties," *IEEE Trans. Ind. Informat.*, vol. 14, no. 6, pp. 2484–2496, Jun. 2018.



Fanghong Guo (Member, IEEE) received the B.Eng. degree in automation science from Southeast University, Nanjing, China, in July 2010, the M.Eng. degree in automation science and electrical engineering from Beihang University, Beijing, China, in January 2013, and the Ph.D. degree in sustainable earth from the Energy Research Institute @NTU, Interdisciplinary Graduate School, Nanyang Technological University, Singapore, in November 2016.

He was a Research Associate and then a Research Fellow with the Rolls-Royce Cooperate Lab @NTU, Nanyang Technological University, from May 2016 to April 2017. From May 2017 to July 2018, he was a Scientist with Experimental Power Grid Center (EPGC), Agency for Science, Technology and Research (A*STAR), Singapore. He is currently with the Department of Automation, Zhejiang University of Technology, Hangzhou, China. His research interests include distributed cooperative control, distributed optimization on microgrid systems, and smart grid.



Bowen Xu received the B.Eng. degree in communication engineering from the Zhejiang University of Technology, China, in 2019, and where he is currently pursuing the M.S. degree with the Department of Automation.

His research interests include optimization, deep learning, and its applications in smart grids.



Wen-An Zhang (Member, IEEE) received the B.Eng. degree in automation and the Ph.D. degree in control theory and control engineering from the Zhejiang University of Technology, China, in 2004 and 2010, respectively.

He has been with the Zhejiang University of Technology, since 2010, where he is currently a Professor with the Department of Automation. He was a Senior Research Associate with the Department of Manufacturing Engineering and Engineering Management, City University of Hong Kong, Hong Kong, from 2010 to 2011. He was awarded an Alexander von Humboldt Fellowship in 2011–2012. He has been serving as a Subject Editor for Optimal Control Applications and Methods since September 2016. His current research interests include networked control systems, multi-sensor information fusion estimation and robotics.



Changyun Wen (Fellow, IEEE) received the B.Eng. degree from Xi'an Jiaotong University, China, in 1983, and the Ph.D. degree from the University of Newcastle, in 1990.

From August 1989 to August 1991, he was a Post-Doctoral Fellow with the University of Adelaide, Australia. Since August 1991, he has been with Nanyang Technological University, Singapore, where he is currently a Full Professor. His main research activities are in the areas of control systems and applications, cyber-physical systems, smart

grids, complex systems and networks.

Dr. Wen was a member of the IEEE Fellow Committee from January 2011 to December 2013 and a Distinguished Lecturer of the IEEE Control Systems Society from 2010 to 2013. He was the recipient of a number of awards, including the Prestigious Engineering Achievement Award from the Institution of Engineers, Singapore in 2005, and the Best Paper Award of the IEEE TRANSACTIONS ON INDUSTRIAL ELECTRONICS in 2017. He is currently the Coeditor-in-Chief of the IEEE TRANSACTIONS ON INDUSTRIAL ELECTRONICS, an Associate Editor of *Automatica* (since Feb 2006), and the Executive Editor-in-Chief of *Journal of Control and Decision*. He also served as an Associate Editor for the IEEE TRANSACTIONS ON AUTOMATIC CONTROL from 2000 to 2002, the IEEE TRANSACTIONS ON INDUSTRIAL ELECTRONICS from 2013 to 2019, and *IEEE Control Systems Magazine* from 2009 to 2019. He has been actively involved in organizing international conferences playing the roles of General Chair (including the General Chair of IECON 2020 and IECON 2023), TPC Chair (e.g. the TPC Chair of Chinese Control and Decision Conference since 2008).



Dan Zhang (Senior Member, IEEE) received the B.E. degree in automation and the Ph.D. degree in control theory and control engineering from the Zhejiang University of Technology, China, in 2007 and 2013, respectively.

He was a Research Fellow with Nanyang Technological University, Singapore, from 2013 to 2014, the National University of Singapore, Singapore, from 2016 to 2017, and City University of Hong Kong, from 2017 to 2019, respectively. His research interests are in the area of networked

control systems, robust control, and filtering.

Dr. Zhang is an Associate Editor of *ISA Transactions*, *International Journal of Uncertainty, Fuzziness and Knowledge-based Systems*, and *International Journal of Control, Automation and Systems*, and was an Associate Editor of *Neurocomputing*.



Li Yu (Senior Member, IEEE) received the B.S. degree from Nankai University, Tianjin, China, in 1985, and the M.S. and Ph.D. degrees in control system and engineering from Zhejiang University, Hangzhou, China, in 1988 and 1999, respectively.

He is currently a Professor in automation with the College of Information Engineering, Zhejiang University of Technology, Zhejiang, China. He has authored or coauthored three books and over 200 journal or conference papers. His current research interests include robust control, time-delay

systems, networked control systems, and motion control.